Branching and workflows

Branching and workflows

- Git and the lab notebook covered the **commit loop**: edit → stage → commit.

Branching and workflows

- Git and the lab notebook covered the **commit loop**: edit → stage → commit.
- This chapter adds **branches**, **tags**, **undo**, and **collaboration** patterns.

What branches are

- A **branch** is a movable **label** on a line of commits.

What branches are

- A **branch** is a movable **label** on a line of commits.
- Default is usually **`main`**.

What branches are

- A **branch** is a movable **label** on a line of commits.
- Default is usually **`main`**.
- Work on a side branch so **`main`** stays shippable while you try ideas.

Create a branch and switch to it

```
git branch
git checkout -b feature/example-task
git branch
```

Create a branch and switch to it

```
git branch
git checkout -b feature/example-task
git branch
```

- Naming: `feature/...`, `bugfix/...`, `docs/...`, `test/...` when it helps others.

Create a branch and switch to it

```
git branch
git checkout -b feature/example-task
git branch
```

- Naming: `feature/...`, `bugfix/...`, `docs/...`, `test/...` when it helps others.
- Dead end? Check out `main` again; delete or keep the branch as you prefer.

Tags as bookmarks

- Tag a commit on a clean tree before a risky experiment.

Tags as bookmarks

- Tag a commit on a clean tree before a risky experiment.

Tags as bookmarks

- **Tag** a commit on a **clean** tree before a risky experiment.

```
git tag v0.2-pre-experiment
```

Tags as bookmarks

- **Tag** a commit on a **clean** tree before a risky experiment.

```
git tag v0.2-pre-experiment
```

- Find the tag later by name or hash when you need that snapshot.

Undoing recent work on a branch

- `git reset --hard HEAD` moves the branch pointer back to the last commit.

Undoing recent work on a branch

- `git reset --hard HEAD` moves the branch pointer back to the **last commit**.
- **Drops** uncommitted work and commits **after** that point on this branch—dangerous if you meant to keep them.

Undoing recent work on a branch

- `git reset --hard HEAD` moves the branch pointer back to the **last commit**.
- **Drops** uncommitted work and commits **after** that point on this branch—dangerous if you meant to keep them.
- Safe only when you understand that nothing unsaved should remain.

Undoing recent work on a branch

- `git reset --hard HEAD` moves the branch pointer back to the **last commit**.
- **Drops** uncommitted work and commits **after** that point on this branch—dangerous if you meant to keep them.
- Safe only when you understand that nothing unsaved should remain.

Undoing recent work on a branch

- `git reset --hard HEAD` moves the branch pointer back to the **last commit**.
- **Drops** uncommitted work and commits **after** that point on this branch—dangerous if you meant to keep them.
- Safe only when you understand that nothing unsaved should remain.

```
git reset --hard HEAD
```

Returning to an older snapshot

- Look up the **hash** or **tag** for the good state.

Returning to an older snapshot

- Look up the **hash** or **tag** for the good state.
- Check out those paths, or reset to that commit/tag by:

Returning to an older snapshot

- Look up the **hash** or **tag** for the good state.
- Check out those paths, or reset to that commit/tag by:
  - `git checkout <commit-hash>`

Returning to an older snapshot

- Look up the **hash** or **tag** for the good state.
- Check out those paths, or reset to that commit/tag by:
  - `git checkout <commit-hash>`
  - `git checkout <tag-name>`

Returning to an older snapshot

- Look up the **hash** or **tag** for the good state.
- Check out those paths, or reset to that commit/tag by:
  - `git checkout <commit-hash>`
  - `git checkout <tag-name>`
- Then **commit** with an honest message about **reverting** or **restoring** with:

Returning to an older snapshot

- Look up the **hash** or **tag** for the good state.
- Check out those paths, or reset to that commit/tag by:
  - `git checkout <commit-hash>`
  - `git checkout <tag-name>`
- Then **commit** with an honest message about **reverting** or **restoring** with:
  - `git commit -m "revert to <commit-hash>"`

Returning to an older snapshot

- Look up the **hash** or **tag** for the good state.
- Check out those paths, or reset to that commit/tag by:
  - `git checkout <commit-hash>`
  - `git checkout <tag-name>`
- Then **commit** with an honest message about **reverting** or **restoring** with:
  - `git commit -m "revert to <commit-hash>"`
  - `git commit -m "restore to <tag-name>"`

Returning to an older snapshot

- Look up the **hash** or **tag** for the good state.
- Check out those paths, or reset to that commit/tag by:
  - `git checkout <commit-hash>`
  - `git checkout <tag-name>`
- Then **commit** with an honest message about **reverting** or **restoring** with:
  - `git commit -m "revert to <commit-hash>"`
  - `git commit -m "restore to <tag-name>"`
- Prefer **branches** for experiments so `main` rarely needs surgery.

GitHub flow

- Team workflow summarized in **GitHub flow**:

GitHub flow

- Team workflow summarized in **GitHub flow**:

GitHub flow

- Team workflow summarized in **GitHub flow**:

1. Branch from the shared default with a **clear name**.

GitHub flow

- Team workflow summarized in **GitHub flow**:

1. Branch from the shared default with a **clear name**.
2. Use a **fork** if you lack write access to the upstream repo.

GitHub flow

- Team workflow summarized in **GitHub flow**:

1. Branch from the shared default with a **clear name**.
2. Use a **fork** if you lack write access to the upstream repo.
3. **Commit** locally; **push** the branch.

GitHub flow

- Team workflow summarized in **GitHub flow**:

1. Branch from the shared default with a **clear name**.
2. Use a **fork** if you lack write access to the upstream repo.
3. **Commit** locally; **push** the branch.
4. Open a **pull request** into the default branch.

GitHub flow

- Team workflow summarized in **GitHub flow**:

1. Branch from the shared default with a **clear name**.
2. Use a **fork** if you lack write access to the upstream repo.
3. **Commit** locally; **push** the branch.
4. Open a **pull request** into the default branch.
5. **Review**; address feedback.

GitHub flow

- Team workflow summarized in **GitHub flow**:

1. Branch from the shared default with a **clear name**.
2. Use a **fork** if you lack write access to the upstream repo.
3. **Commit** locally; **push** the branch.
4. Open a **pull request** into the default branch.
5. **Review**; address feedback.
6. **Merge** when accepted.

GitHub flow

- Team workflow summarized in **GitHub flow**:

1. Branch from the shared default with a **clear name**.
2. Use a **fork** if you lack write access to the upstream repo.
3. **Commit** locally; **push** the branch.
4. Open a **pull request** into the default branch.
5. **Review**; address feedback.
6. **Merge** when accepted.
7. **Delete** the merged branch.

Example workflow

1. Create a new branch

```
git checkout -b small-edits
```

Example workflow

1. Create a new branch

```
git checkout -b small-edits
```

2. Make changes, commit frequently

```
git add my_new_file.py
git commit -m "add data-loading script"
```

```
git commit -a -m "fix column header in loader"
```

Example workflow

1. Create a new branch

```
git checkout -b small-edits
```

2. Make changes, commit frequently

```
git add my_new_file.py
git commit -m "add data-loading script"
```

```
git commit -a -m "fix column header in loader"
```

3. Push the branch

```
git push -u origin small-edits
```

Another example workflow

1. Sync with the remote and branch

```
git checkout main
git fetch --all --prune
git rebase
git checkout -b bugfix
```

Another example workflow

1. Sync with the remote and branch

```
git checkout main
git fetch --all --prune
git rebase
git checkout -b bugfix
```

2. Fix and push

```
git commit -a -m "fix off-by-one in scoring"
git push -u origin bugfix
```

Another example workflow

1. Sync with the remote and branch

```
git checkout main
git fetch --all --prune
git rebase
git checkout -b bugfix
```

2. Fix and push

```
git commit -a -m "fix off-by-one in scoring"
git push -u origin bugfix
```

3. Open a **pull request** on GitHub; merge after review.

Git habits

- **Commit** often — small steps are easier to understand and revert.

Git habits

- **Commit** often — small steps are easier to understand and revert.
- **Pull** latest changes before starting new work.

Git habits

- **Commit** often — small steps are easier to understand and revert.
- **Pull** latest changes before starting new work.
- **Branch** for every task; keep `main` clean.

Git habits

- **Commit** often — small steps are easier to understand and revert.
- **Pull** latest changes before starting new work.
- **Branch** for every task; keep `main` clean.
- **Write** descriptive commit messages (commit discipline).

Git habits

- **Commit** often — small steps are easier to understand and revert.
- **Pull** latest changes before starting new work.
- **Branch** for every task; keep `main` clean.
- **Write** descriptive commit messages (commit discipline).
- **Test** changes before committing.

Quick command reference

Command	Purpose
`git status`	Show working-tree state
`git add <path>`	Stage files
`git commit -m "…"`	Record staged snapshot
`git log`	Browse history
`git diff`	Compare working tree to last commit
`git branch` / `git checkout -b`	List / create branches
`git push` / `git pull`	Sync with remote
`git fetch`	Download remote changes without merging
`git merge`	Combine branches
`git clone`	Copy a remote repo locally

Summary

- **Branches** isolate experiments; merge when ready.

Summary

- **Branches** isolate experiments; merge when ready.
- **Tags** mark important snapshots; **reset** undoes recent missteps.

Summary

- **Branches** isolate experiments; merge when ready.
- **Tags** mark important snapshots; **reset** undoes recent missteps.
- **GitHub flow**: branch → commit → push → pull request → review → merge.

Summary

- **Branches** isolate experiments; merge when ready.
- **Tags** mark important snapshots; **reset** undoes recent missteps.
- **GitHub flow**: branch → commit → push → pull request → review → merge.
- Build the habit: **small commits, clear messages, frequent sync**.

Resources

- GitHub flow

Resources

- GitHub flow
- Git tutorial · Git cheat sheet (PDF)

Resources

- GitHub flow
- Git tutorial · Git cheat sheet (PDF)
- Ubuntu CLI cheat sheet (PDF) · Linux-fu

Resources

- GitHub flow
- Git tutorial · Git cheat sheet (PDF)
- Ubuntu CLI cheat sheet (PDF) · Linux-fu
- Russell A. Poldrack, Better Code, Better Science – book home

Resources

- GitHub flow
- Git tutorial · Git cheat sheet (PDF)
- Ubuntu CLI cheat sheet (PDF) · Linux-fu
- Russell A. Poldrack, Better Code, Better Science – book home
- The curious coder's guide to git